Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model

Yining Cao
University of California, San Diego
San Diego, California, USA
yic069@ucsd.edu

Peiling Jiang
University of California San Diego
San Diego, California, USA
peiling@ucsd.edu

Haijun Xia
University of California, San Diego
San Diego, California, USA
haijunxia@ucsd.edu

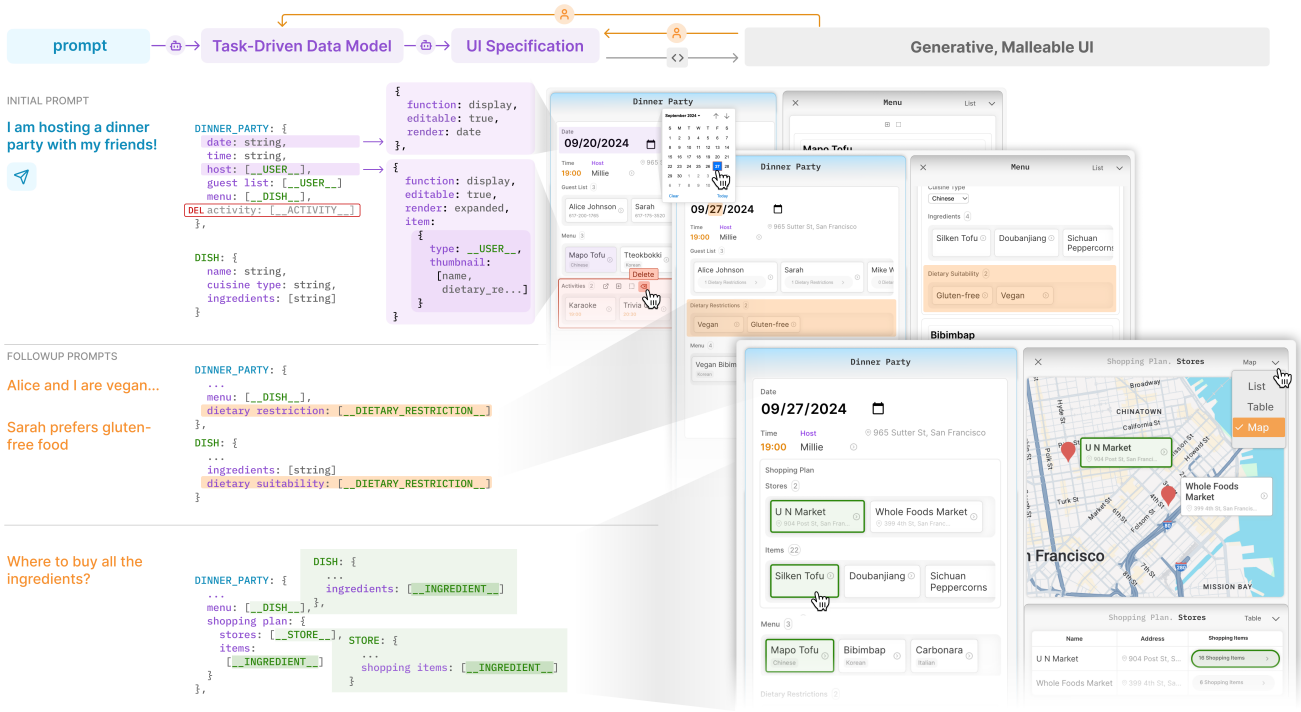


Figure 1: We propose an approach towards generative and malleable user interfaces. With the evolving task-driven data models as foundation, the interface can be generated and dynamically evolves with users' changing information needs.

Abstract

Unlike static and rigid user interfaces, generative and malleable user interfaces offer the potential to respond to diverse users' goals and tasks. However, current approaches primarily rely on generating code, making it difficult for end-users to iteratively tailor the generated interface to their evolving needs. We propose employing task-driven data models—representing the essential information entities, relationships, and data within information tasks—as the foundation for UI generation. We leverage AI to interpret users' prompts and generate the data models that describe users' intended tasks, and by mapping the data models with UI specifications, we can create generative user interfaces. End-users can easily modify and extend the interfaces via natural language and direct manipulation, with these interactions translated into changes in the underlying model. The technical evaluation of our approach and user evaluation of the developed system demonstrate the feasibility and effectiveness of the proposed generative and malleable UIs.

CCS Concepts

• **Human-centered computing** → **User interface programming**; **HCI theory, concepts and models**.

Keywords

Generative User Interface, Malleable User Interface

ACM Reference Format:
Yining Cao, Peiling Jiang, and Haijun Xia. 2025. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model. In *CHI Conference on Human Factors in Computing Systems (CHI '25), April 26–May 01, 2025, Yokohama, Japan.* ACM, New York, NY, USA, 20 pages. https://doi.org/10.1145/3706598.3713285

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

CHI '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1394-1/25/04
https://doi.org/10.1145/3706598.3713285

1 Introduction

The vision of personalized and intelligent user interfaces, as portrayed in Apple's 1987 Knowledge Navigator [6], seems more attainable than ever given the recent advances in AI [2, 4]. We envision the interfaces to be capable of responding to users' diverse requests, and continuously adapting to users' evolving needs by presenting relevant information with effective representations and interactions. A critical challenge in realizing this vision is devising an interface paradigm and technical approach for creating such generative and malleable user interfaces.

Consider the task of hosting a dinner party. One needs to fix the schedule, invite guests, plan the dishes, compare wine options, finalize a shopping list, and determine the optimal shopping route. Under the dominant application-centric interface paradigm, end-users need to cobble together a large number of applications, using a fraction of the functionality of each to accomplish their goals [14, 72]. This fragmented, inefficient workflow is a common experience in our everyday informational tasks. While one could imagine dedicated applications developed for different tasks, it is impractical given the diversity of user needs. In this case, a dedicated "dinner party" application may not exist, and if it did, it would likely become bloated with features while still failing to fully accommodate individual preferences and evolving requirements.

The programming capability of generative models offers one approach to achieve generative and malleable UIs: generating the codebase of a custom application from user prompts, which could be compiled and executed to support users' tasks [4, 49, 69]. However, the code-generation approach makes it challenging for end-users to modify and extend the interfaces when AI's generation inevitably fails to fully align with users' needs or when those needs naturally shift throughout the task. Each new prompt-based revision may result in a discontinuous transition between generated codebases, making it difficult to maintain consistency across iterations; it is also unclear how the data should be transformed when the user's tasks require changing the underlying information structure. The opaque relationship between user prompts and the resulting code further complicates interpretability and control, limiting end-users' ability to steer the generation process effectively.

A potential solution is to introduce higher-level generative structures that can guide both UI generation and data transformations while improving end-user control. This work explores such structures to generate interfaces that are both malleable and interpretable.

We adopt the canonical perspective that user interfaces are graphical representations of underlying data models that describe the intended tasks, rooted in the Model-View-Controller (MVC) framework for GUI-based applications [27] and model-based UI development [37, 50, 57]. In this view, traditional applications employ *fixed models* for *predefined tasks*, and therefore, the onus is on the users to piece together the separate application models to match their workflows. On the contrary, generative and malleable UIs should dynamically evolve to reflect users' tasks and intentions. To achieve this, we propose leveraging Large Language Models (LLMs) to interpret users' prompts and generate a *task-driven data model*—a structured representation of the essential entities, relationships, and data properties relevant to the intended task. This model serves as the foundation for generating UI specifications that define the components and composition of the interface. The model will evolve continuously in response to users' changing needs. As such, the dynamically generated and evolving task-driven data models can drive the transformation of the interface and the underlying data, achieving generative and malleable UIs.

This research investigates the feasibility of this approach by exploring both the technical pipeline and interaction techniques of the generative and malleable UIs with a prototype system, Jelly. The pipeline begins by analyzing user prompts and generating a model that consists of an object-relational schema and a dependency graph. This model then guides the generation of the UIs by representing aspects of the model with predefined UI patterns and rules that reflect common UI design practices [60]. Within Jelly, users can interact with the generated interfaces using natural language and direct manipulation, with these interactions translated to changes in the underlying model. Users can also directly inspect the model to understand the underlying structure of the interface and flexibly customize it to suit their needs.

We evaluate the LLM-generated data model with a technical evaluation. Results show that state-of-the-art LLMs can reliably generate relevant entities and dependencies that can meet the information and interaction needs expressed in users' natural language prompts. To the best of our knowledge, the generative and malleable UIs we developed are the first of their kind to support relatively open-ended information tasks. To assess their effectiveness, we conducted a user study where participants engaged with the system to complete several open-ended information tasks and reflected on how their experiences compared to those with existing GUI applications and AI-powered chat interfaces. Our findings show that generative and malleable UIs enabled users to develop highly personalized and dynamic information spaces by flexibly curating diverse information and customizing how information is presented.

Therefore, this work makes the following contributions:

- A technical approach for generative and malleable UIs based on the generative task-driven data models that evolve with users' tasks.
- Jelly, a prototype system that implements the proposed approach, enabling users to flexibly generate and customize user interfaces via natural language and direct manipulation.
- Technical and user evaluations demonstrating the feasibility of the approach and the strengths and future directions of such generative and malleable interfaces.

2 Research Framing and Scope

Our long-term goal is to develop dynamic, personalized, and adaptive interfaces that cater to individuals' unique and evolving needs across various domains, scopes, and levels of complexity. Such interfaces are particularly well-suited for information tasks that require integrating data from multiple domains, presenting information in highly personalized ways, and supporting open-ended or exploratory workflows where users' goals and information needs continuously evolve. These types of tasks include but are not limited to planning tasks (e.g., travel and party planning), exploring and learning about a new domain (e.g., red wine, cheese), or multi-factor decision-making tasks (e.g., holiday gift shopping, choosing colleges to attend).

2.1 Research Scope

For generative and malleable UIs to effectively support users' information tasks with our model-based approach, the following key aspects need to be addressed: (1) *Task Representation.* The model needs to employ an effective representation to describe users' information tasks; (2) *UI Generation.* The model needs to be translated into rich and effective interface representations; (3) *Model Evolution.* The model needs to dynamically adapt to users' evolving tasks and various customization needs. (4) *Data Integration.* The model needs to be populated with accurate and relevant data. (5) *Context Awareness.* The interface should incorporate users' context to provide personalized information and UI configurations.

In this paper, we focus on the first three aspects. While fully addressing all aspects is beyond our scope, our technical pipeline and prototype system presented in this work provide a clear guide for future research to continue advancing this paradigm, which we will discuss in detail in Section 9. Below, we describe an example scenario of using Jelly. In this scenario, we assume Jelly has access to the user's personal information.

2.2 Envisioned Scenario

Millie is planning to host a dinner party with her friends. Typically, she would have to use the browser to search dishes online, find their recipes, use a note-taking application to record the ingredients and make a shopping plan, use a calendar and several communication applications to coordinate the schedule with her friends, and more.

Instead of juggling all these applications, she opens Jelly and types, "*I am hosting a dinner party.*" Jelly responds with a few follow-up questions with generated GUIs, such as "who to invite" with a list of selectable contact cards; and "when the party is" by presenting a calendar populated with her schedule. After a brief conversation, Jelly generates a home panel for the "Dinner Party Plan" task, including the time, location, guest list, menu, and activities. Seeing this, Millie realizes that she has forgotten to invite a few people. With Jelly, she can pull up her contact list by clicking on the "all" button beside the guest list. The panel of all her contacts is displayed side-by-side. Contacts already added to the guest list are highlighted in both the guest list and the contact list. Millie then adds the missing guests by tapping on their contact cards.

Millie reviews the recommended dishes in the menu, deletes the ones she dislikes, and clicks the "add" button next to the menu list to explore more options suggested by Jelly. While doing so, she realizes that she needs to consider dietary restrictions. She informs Jelly, "*Alice and I are both vegan.*" To fulfill this request, Jelly adds a "dietary restrictions" attribute for all guests and automatically records Alice's and Millie's preferences. Meanwhile, a "dietary suitability" attribute has been added for each dish, flagging dishes violating the dietary restrictions. Millie then replaces them with suggestions made by Jelly. To ensure awareness of the restrictions when planning the activity, Jelly also adds a new section to the home panel summarizing the dietary restrictions of all guests.

When the menu is finalized, Millie then types "*I need to get the ingredients.*" Recognizing the task shift, Jelly generates a Shopping List panel, organizing the ingredients as shopping items with attributes assisting the ingredient purchasing process: The "total quantity" aggregates the amount needed for all the dishes; the "store" drop-down menu lists all the local stores where the ingredients are available; and the "bought" checkbox tracks the shopping progress. Jelly presents the stores in a map view for her to plan the shopping trip. Clicking on each store on the map shows her all the items that she needs to buy at the store. After reviewing the list, she clicks the "start" button at the bottom of the map, hops in her car, and heads out to shop.

3 Related Work

We adopt the canonical perspective that user interfaces employ interactive graphical representations to encode the underlying data model, similar to the Model-View-Controller (MVC) software design framework for GUI-based applications [27], and the model-based UI development paradigm [37, 57]. Taking this perspective, traditional GUI applications employ fixed data models and fixed encodings (i.e., program) to create fixed interfaces, resulting in rigid applications designed for specific tasks with specific features. Extensive research has explored various approaches towards the creation of adaptive, dynamic, malleable, and generative UIs. In what follows, we review the key approaches that have been explored.

3.1 Model-Based User Interfaces

Model-Based User Interfaces (MBUI) development arose as a paradigm that aims to significantly reduce the effort in developing UIs while ensuring quality [37, 39], initiated by early works on User Interface Management System (UIMS) [38, 47] that proposed decoupling application functionality from the UI. MBUI provides a systematic approach to software design by leveraging abstract models, including task models to structure task workflows, domain models to represent data relationships, and abstract-to-concrete mappings to render the UI components. Utilizing these models, developers can specify interfaces declaratively at a higher level of abstraction [25, 57]. For example, rather than concretely specifying the UI components, such as a set of radio buttons or a dropdown list, MBUI tools allow developers to define their needs declaratively, such as "a widget for selecting a single item from a set," so the system can decide the most appropriate widget to display based on the specific user scenario, for example, screen sizes of the devices.

Prior works have explored supporting different subprocesses and UI scopes for MBUI. For example, UIDE focuses on dialog box generation by assigning data types to widgets and laying them out on a canvas [13], while MIKE generates menus and dialog boxes directly from function signatures [48]. MASTERMIND [59] and ITS [67] expand on these approaches by supporting a broader range of interface specifications. Automating the mapping from high-level models to UI specifications has also been a significant focus in prior research. For instance, TRIDENT balances automation with manual refinement, allowing developers to specify presentation and navigation strategies [65]. Despite these advancements, one persistent challenge in MBUI is translating the abstract models into concrete UI components. Systems like HUMANOID [34, 58] and BOSS [55] use reusable templates to address this issue. TIMM further generalizes these solutions into a computational framework that explicitly represents and manages the mappings between abstract and concrete elements [50].

Our approach takes a similar perspective of separating interface presentation from the underlying system logic that is governed by task-driven data models, and explores automating the process of encoding and mapping between these layers. The primary focus of MBUI, however, has been on assisting developers in creating UIs rather than enabling end-users to modify their interfaces dynamically. Therefore, the models that the MBUI approach produces are predefined and static. Our work, on the other hand, aims to continuously update the underlying model, which drives the transformation of user interfaces to meet the end-users' evolving needs.

3.2 Specification-Based UI Generation

Different from the traditional MBUI development paradigm, with which software developers determine the underlying task model, UI model, and their mappings to create a single system [50, 67], the specification-based UI generation can be seen as a scaffolded approach of MBUI such that interfaces can be specified by end-users or automatically generated. This is typically achieved by constraining one or more aspects within the MBUI approach. For example, Bespoke relies on the specification of command-line applications, and by predefining the mappings between UI widgets to different types of command-line parameters, it enables end-users to create GUIs for command-line applications through demonstration [64]. Similarly, DynaVis leverages the Vega-Lite specification [53] to generate visualization editing interfaces. By mapping UI widgets to visualization parameters, it composes appropriate UI components based on parameters inferred from users' natural language queries [63]. To ensure the quality and consistency of UIs for controlling home appliances, Nichols et al. employed a specification language and parameterized templates that encode design conventions, enabling the automatic generation of structured and coherent interfaces [43, 44].

A unique strength of high-level specification is that it enables developers and end-users to focus on composing high-level domain-specific primitives, delegating low-level execution to the underlying architecture and runtime [19]. By choosing an appropriate level of abstraction and enabling one-to-one mappings between specification and user interface components, end-users can often directly manipulate the specification itself to adjust the generated outcome. Additional interface layers, such as graphical or natural language interfaces, can also be utilized on top of the specification if fully instantiating the specification is tedious [41, 63, 64].

In this approach, domain experts define the specifications, shaping the UI generation space within a structured yet adaptable framework. While high-level specifications impose constraints compared to low-level programmatic approaches, they improve accessibility by making the entire UI generation space more interpretable and modifiable by end-users. Moreover, these constraints help maintain design consistency and quality across generated interfaces [44]. Building upon this approach, we developed a set of UI specifications to translate the data model into interface representations.

3.3 End-User Programming/Development

Prior research has explored creating interactive systems that are customizable by end-users, allowing them to tailor the interface to suit their specific needs. This body of work is primarily situated within the field of end-user programming or development (EUP or EUD) [40], where end-users can employ natural language programming [29], GUI-based interaction [8, 32, 75], visual programming [51, 66], and programming-by-demonstration [30] to extend existing systems.

For example, pioneering systems like OpenDoc [7], HyperCard [8], Smalltalk [24], and recent systems, such as DynamicLand [66] and Embark [56] aimed to develop dynamic and personal media for end-users to create their own dynamic content and UIs. Hypercard allows users to develop interactive multimedia content by linking objects via GUI and scripting advanced behaviors using a built-in programming language [8]. These systems pre-define the underlying data model but expose the encoding mechanism to enable end-user development.

Most existing systems, however, do not expose their data models and encoding mechanisms to the end-users. To circumvent this, research has explored constructing external data models and associated encoding mechanisms to enable EUD. For example, Wildcard leverages the accessible and manipulable Document Object Model (DOM) of webpages to enable end-users to collect data from and inject data back into the DOM structure. By representing webpage data in an external spreadsheet, users can manipulate the spreadsheet to customize web pages [32]. Further leveraging the composability and transclusion of the web [42], prior work explored enabling end-users to create mashup applications tailored to their specific needs. For example, Fusion [75] and C3W [15] enable users to create mashups by extracting components from existing webpages and connecting them using transclusion, formula, and glue code. Vegemite allows users to collect data from multiple websites; using scripts that can be generated from users' demonstration, it can perform computation on the collected data and automatically execute web actions such as clicking links and inputting data values to web forms [30]. In cases where DOM-like accessibility is unavailable, Research has explored reverse-engineering to extract useful structure and metadata from UI elements. For example, Prefab explores recognizing UI widgets on any GUI applications, and then modifying their behaviors using input and output redirection [12]. Zhang et al. developed machine learning models to extract metadata from UI screens, which can be used to enhance the screen's accessibility [74].

While EUP/EUD allows users to extend applications, they lack direct access to the applications' internal data models. While external data models (e.g., spreadsheet, recognized interface structures) can serve as the proxies or connectors with the original applications, these external data model are also pre-defined by developers, leaving end-users with limited customizability.

3.4 Context-Based UI Adaptation

Prior work has also explored computationally adapting UIs based on various contextual factors and constraints pertinent to the device, user, or situation in domains such as accessibility, ubiquitous computing, and mixed reality [1, 11, 17, 22, 68]. For example, SUPPLE and SUPPLE++ computationally adjust the size, style, and layout of widgets to adapt the user interfaces based on device constraints (e.g., screen size, input modality) and users' capabilities (e.g., motor

and vision) [16–18]. UNIFORM [45] and Huddle [46] automatically generate UIs with primitive widgets for controlling home appliances by considering users' interaction history as well as modeling the similarities and dependencies of appliances. Since mobile and Mixed Reality (MR) interfaces can be invoked in arbitrary situations, research has also explored adapting these interfaces based on various environmental factors [11, 31]. For example, Lindlbauer et al. explored how applications should adapt the amount of information they show and their spatial arrangement in MR[31].

With this approach, the data models of the interactive systems are often extended with developer-defined constraints, which will take effects with anticipated contextual input, resulting in context-aware dynamic interfaces. However, the scope of the dynamic behaviors is pre-defined by developers. Therefore, the adaptability—both in terms of *what* and *how* to adapt—is often less controllable by end-users.

3.5 AI-based Code Generation

Recent developments in AI, especially its ability to generate functional program code from natural language prompts, have sparked a new approach towards generative UI. AI products such as Claude [4] and Vercel [49] can generate and render UI code from natural language prompts. Wu et al. explored fine-tuning LLMs with automated feedback to improve the quality of the generated UI code [69]. However, they found that the state-of-the-art AI models could struggle to reliably produce compilable programs (less than 80% compilation success for a single UI screen).

As AI's code generation capability continues to improve [4], the complexity and quality of AI-generated applications and UIs are expected to increase. This approach presents both opportunities and challenges. On one hand, AI can generate code from arbitrary user requests, making it a more scalable approach for UI generation. On the other hand, the inherent entry barriers associated with programming languages and tasks as well as the opaque mappings between natural language prompts and generated code create significant challenges for end-users in understanding, controlling and customizing the output [26, 70]. Given AI's inconsistent performance—even in generating single-screen UIs—it remains unclear how AI-based code generation can reliably and continuously adapt interfaces to meet users' dynamic and shifting goals. Additionally, current AI-driven code generation approaches primarily focus on generating code for client-side UIs, leaving open challenges on how the server-side data should be structured and transformed [49, 69].

As a known issue observed across many domains of AI-generated content, creating and iterating via prompting is inherently challenging [33, 52, 70, 73]. To address this, additional high-level control structures are often required. Therefore, it is important to devise high-level structures to guide the generation process, such as imposing additional conditioning constraints for image generation [73] and leveraging compositional structures to ground video generation [10]. In this work, we take a similar approach by introducing task-driven data models—a high-level control structure that guides UI generation and enables users to more easily inspect and adjust the generated interfaces.

4 Design Goals

To achieve the envisioned generative and malleable UIs that can respond to and evolve with users' information tasks, we build upon the existing endeavors and propose the following design goals:

DG1 Developing Effective Task-Driven Data Model to Represent Users' Information Tasks. To support users' information tasks with effective UIs, the underlying foundation–the data model–must be able to effectively represent the information tasks. Unlike some traditional models in MBUI that prescribe the interaction and task sequences [21], which can lead to rigid workflows, we will design the model to represent the essential entities, relationships, and constraints needed to accomplish the information tasks, allowing users to form their own workflows. Additionally, we aim to design the model in a way that it can be intuitively interpreted and manipulated.

DG2 Translating the Task-Driven Data Model into Effective UIs with UI Specification. With the model as the foundation, we aim to take the specification-based approach to ensure the consistency and expressiveness of the generated UIs. To effectively map the abstract models to concrete UIs, the specification should be grounded based on a set of common design patterns that describe what UI widgets should be used and how they should facilitate interaction with different types of information. We aim to use generative AI for generating the specifications; therefore, the specifications should be designed to be effectively leveraged by AI for robust and accurate generation of user interfaces.

DG3 Providing Interactions for End-Users to Modify the UIs to Align with Their Evolving Needs. To accommodate various interaction modalities and levels of specificity, we aim to empower end-users to express their intended tasks and UI modifications through both natural language prompts and direct manipulation. These interactions will be translated into updates to the underlying model. In addition, we plan to provide an "Inspect"-like tool, similar to browser developer tools, for end-users to directly examine and edit the model for enhanced interpretability and control over both the generation process and resulting interfaces.

5 Technical Pipeline for Generative and Malleable User Interface

Building upon the design goals outlined above, we propose a technical pipeline that takes users' prompts as input and generates corresponding user interfaces, as illustrated in Fig. 2. The pipeline begins by analyzing user prompts to infer user goals and derive sub-tasks. This information is then leveraged by LLMs to generate the **Task-Driven Data Model**, which represents the structure of the task. The data model is then translated into a **UI Specification** that defines the composition of UI elements and manages their states. Users can continuously provide natural language prompts and directly manipulate the generated interfaces, which are both translated into corresponding changes to the underlying data model, and subsequently drive real-time updates to the underlying data and/or UIs, resulting in generative and malleable user interfaces. We detail each component of the pipeline in the following sections.

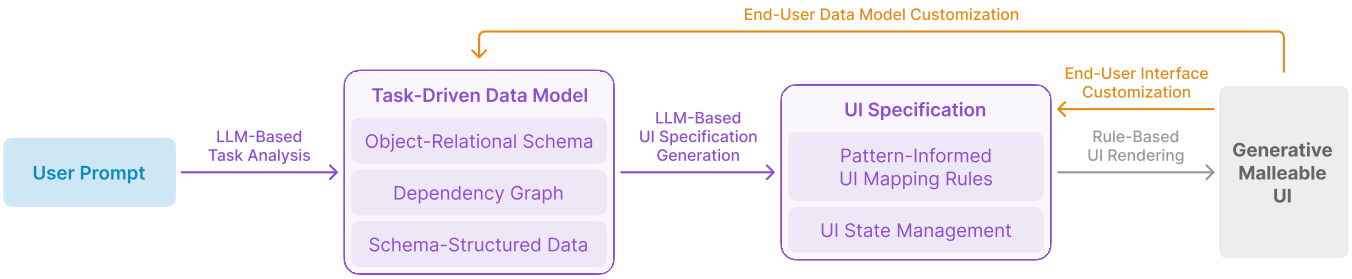


Figure 2: Our proposed pipeline for generative and malleable UIs. The pipeline takes the user prompt as input and employs LLMs to generate a data model describing the task. This model serves as the foundation for generating UI specifications, which guide UI composition and state management. The specifications are then used to render the UI based on predefined rules. Users can iteratively customize both the data model and UI specifications through interactions.

5.1 Task-Driven Data Model Generation

The task-driven data model comprises three components: (1) the *Object-Relational Schema* that describes the types of entities required by the task, as well as their attributes and relationships; (2) the *Dependency Graph* that describes additional dependency relationships across entities, and (3) *Structured Data* that instantiates the schema and dependencies with concrete values.

5.1.1 Object-Relational Schema. We model the user task with an object-relational schema, with which the task and its entities are represented as objects with attributes, and relationships among entities are modeled as references among the objects. Fig. 3 shows a sample schema generated with the prompt—"*give me a weekly meal plan.*" A schema consists of the following elements:

Task. The task object is the root of the object-relational schema, which describes the attributes essential to the overall task. For example, the task object of a travel planning task might include attributes such as destination, duration, and itinerary. A meal plan task object might include start/end dates and a daily plan (Fig. 3a).

Entity. The schema contains entities that model the essential components of a task. For example, the task of creating a meal plan consists of entities such as daily meal plans, recipes, ingredients, and grocery stores (Fig. 3b). In another case, a literature review task might include entities such as paper and author. Each entity contains attributes and cross-references with other entities.

Attribute. Attributes of the task and entity objects are rendered based on their data types, which can be one of the four types:
`SVAL` is a singular data value, such as date, location, etc. (Fig. 3c$_1$).
`DICT` is a dictionary that stores key-value pairs, such as the nutrition facts for a dish entity (Fig. 3c$_3$).
`PNTR` is a reference to another entity, such as a pointer to a "store" entity in a shopping item (Fig. 3c$_4$).
`ARRY` is a collection of items of `SVAL` or `PNTR` type (Fig. 3c$_2$). Note that schema syntax does not allow array of `DICT`. If there are multiple entities that share the same `DICT`, they will be abstracted as an entity, and their references will be treated as `PNTR`. This abstraction simplifies the data model and ensures consistency in how entities and attributes are handled across the system.

5.1.2 Dependency Graph. Dependencies are an essential aspect of complex tasks, which manifest in the UI as relationships between components. Our pipeline uses LLMs to generate these dependencies based on the characteristics of the task, expressed as:

$$\textit{Dependency} := \{\textit{Source}, \textit{Target}, \textit{Mechanism}, \textit{Relationship}\} \quad (1)$$

Source and *Target* refer to specific entities or attributes within the object-relational schema.

Mechanism defines how the target reacts to the changes of the source in one of the following two ways: `Validate` ensures constraints are upheld. For example, the checkout date must be later than the check-in date. If violated, the update of the checkout date value will be rejected, and the UI will highlight the issue to explain the violation to the user. `Update` automatically propagates changes. For example, the total calories of a dish update automatically if the quantities of the ingredients change.

Relationship defines the relationship between *Source* and *Target*. A JavaScript snippet will be generated if the dependency can be expressed by code, e.g., numerical calculations or validations. Otherwise, the relationship is described in natural language, which LLMs can process to apply the effects.

5.1.3 Structured Data. Once the schema and dependency graph are defined, the next step is to acquire data that conforms to the specified structure and constraints. The pipeline is designed to support real-time data integration from multiple sources, handling both structured and unstructured data, such as generated data, user-uploaded data, and external APIs (e.g., TripAdvisor for travel [61] or Semantic Scholar for research [54]). Since our focus is on evaluating the generation of the data model and the user interface, the current pipeline primarily relies on generated data. However, future work could extend its capabilities to incorporate live data acquisition, which we further discuss in Section 9.4.

5.2 UI Specification Generation

With the task-driven data model, the pipeline then generates the user interface based on the model. To ensure consistency, stability, and quality of the generated UIs, we opted for a specification-based approach explored in prior work [53, 63, 64]. Therefore, this step of the pipeline takes the data model and generates the UI specification, which will guide the composition of the user interface.

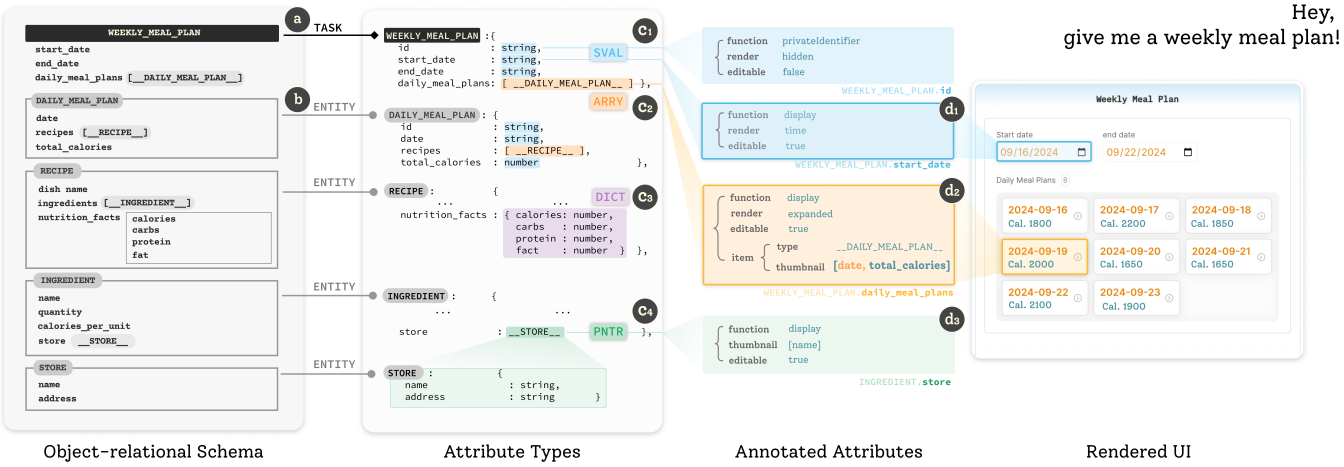


Figure 3: The format of object-relational schema and the annotation of attributes within the schema. (a) task attributes of the schema, (b) entities described in the schema; (c1) single data value attribute, (c2) array attribute, (c3) dictionary attribute, (c) pointer attribute; (d1) annotated single data value, (d2) annotated array attribute, (d3) annotated pointer attribute

5.2.1 Annotating Object-Relational Schema with UI Mapping Rules. Specifically, the pipeline examines each task and entity attribute and annotate each with labels that specify their data types, function roles, and rendering types. The annotations serve as a specification that guides the mapping of schema elements to UI components using a rule-based approach. We provide the full specification and an example in Appendix A.

Attributes of DICT, PNTR, ARRY may take significant screen space if fully rendered. Therefore, care needs to be taken to ensure the appropriate amount of information is presented on the interface through appropriate UI composition to enable progressive disclosure. Below, we describe how each type of attribute will be labeled and how the labels will affect view composition.

SVAL is labeled with `<function, render, editable>` to describe the functional role, the corresponding rendering widget type (e.g., text, time, or location), and if the user may change the value within the GUI. For example, in Fig. 3d_1, the `start_date` is labeled as `<display, time, true>`, which will be rendered as a calendar widget on the interface that can receive user edits.

DICT itself is not labeled, but all attributes within it will be labeled and directly rendered within DICT attribute's parent view.

PNTR is labeled with `<function, thumbnail, editable>` with `<function, editable>` the same as SVAL . `<thumbnail>` specifies the attributes in the referred entity that should be displayed for each minimized item. For example, in Fig. 3d_3, the `store` attribute for every ingredient is a pointer to a store entity. When rendering the ingredient item on the UI, it will only show the name of the store as a hyperlink to the full details of the store it points to.

ARRY is labeled with `<function, render, editable>`. The `render` type for an ARRY can be "expanded" or "summary". When labeled with "expanded", the list will be fully rendered. When labeled with "summary", the list will be rendered in a minimized format, only showing the designated summarizing text and corresponding value, e.g., "Total Calories 2100" for a list of dishes. The summarized form, upon clicked, can expand and show the full list. We further illustrate the interaction mechanisms used for navigating these collections of objects in Section 6.1.2.

5.2.2 Executing Dependency Graph with UI State Management. The generated dependency mechanisms are executed with corresponding UI state management rules, ensuring that Jelly reliably and consistently handles the logic for the generated UI. The state management unit of the system sandboxes each dependency execution to limit its effects to ensure UI stability, interprets, and executes the updating or validating mechanisms accordingly.

5.2.3 UI Rendering. With the specification, the UI rendering process starts with the object-relational schema for the overall task and recursively renders each referred entity and its attributes. Mapping from the specification labels to the UI widgets is handled through a predefined set of rules that ensures consistency across model and data updates. We included the mapping rules used in the current prototype in the supplemental materials.

5.3 Customization with Continuous Prompting

As users provide follow-up requests to Jelly, the pipeline dynamically updates both the data model and the UI. The system leverages previous prompts and data models as context, querying the LLM to determine the necessary update operations. It first assesses whether the request requires modifications to the schema (e.g., adding, removing, or updating entities or attributes) and/or updates to the data. These requests are then parsed into a sequence of operations specified as:

$$\textit{Updater} := \{\textit{Target}, \textit{Action}, \textit{Specifications}\} \quad (2)$$

The *Target* refers to the path of the relevant entities or attributes. *Action* includes operations such as add, remove, and update (for both schema and data), as well as data-specific operations like cluster, filter, and sort. *Specifications* details the specific changes to be made for the given action, such as the name of the attribute to be added to the target entity schema. Based on this, the LLM generates the necessary operations to update the UIs.

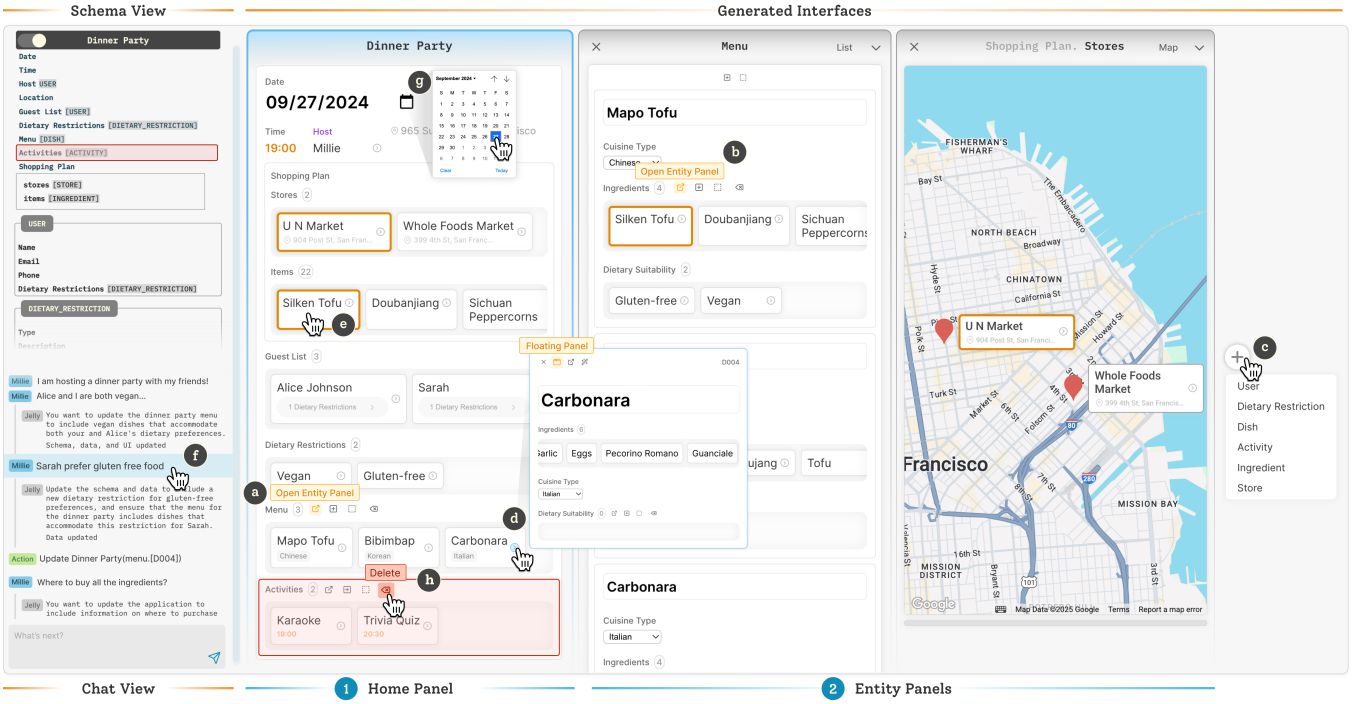


Figure 4: Jelly consists of the Schema View, the Chat View, and the Generated Interfaces. The task object is presented as the Home Panel (1). Users can navigate to Entity Panels (2) by expanding items in other panels (a-b) or using the navigation button on the right side of the interface (c). Users may click to reveal details of an item (d) and hover over an item to trigger synchronized highlighting across panels (e). Users can customize the UIs and data through continuous prompting (f) and direct manipulation (g-h), with all actions preserved as a traceable history in the Chat View.

5.4 Implementation

Our current pipeline is developed using Python, with the front-end developed in JavaScript using the React framework. To optimize performance, we process independent pipeline requests to LLMs concurrently. For the generation steps, we leverage Anthropic's Claude 3.5 Sonnet and OpenAI's GPT-4o models, selected based on internal performance testing across various pipeline tasks. At each generation step, we incorporate the user's previous request as context. The LLMs are instructed to first infer tasks implied by the user's current prompt, then generate a JSON object in a specified response format. To ensure controlled and accurate outputs, we use few-shot prompting tailored to each step. Additionally, the pipeline performs compatibility checks on the generated schema and data before rendering. Full implementation details and service usage are provided in the supplemental materials.

6 Jelly: a Generative and Malleable Interactive System

Jelly is a prototype system developed using the technical pipeline described above (Fig. 4). Users enter Jelly with a prompt that specifies their task. In addition to the main generated interfaces, Jelly's sidebar comprises a Schema View, which surfaces the object-relational data schema; and a Chat View that allows users to provide follow-up prompts, where Jelly responds in natural language about interface changes. In the following sections, we describe the interface designs and interaction techniques in Jelly for effectively supporting users' tasks and customization of the generated UIs.

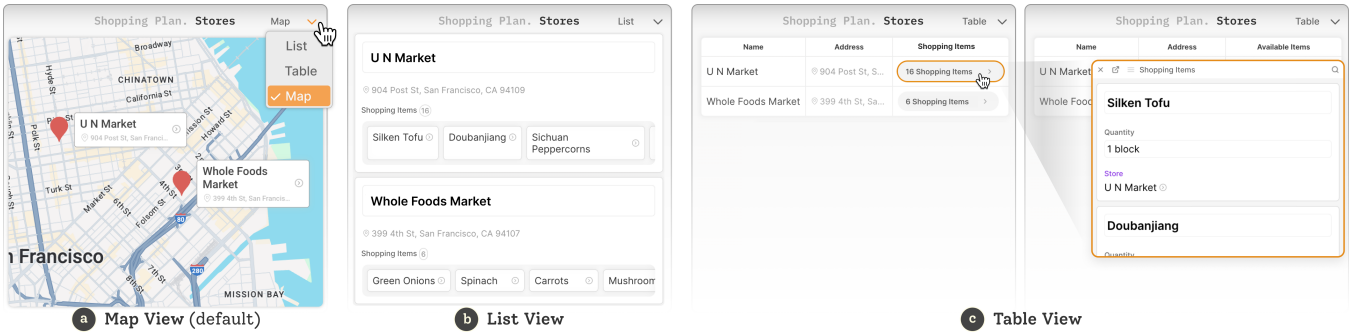
6.1 View Management

User tasks are often supported by complex data models, therefore, presenting all information in a single view can be overwhelming. Jelly employs a set of view management strategies that help users comprehend, navigate, and interact with the generated interfaces.

6.1.1 Panels for Showing Task and Entity Objects. The initially generated interface displays only the Home Panel (Fig. 4-1), which corresponds to the top-level task object. This provides users with a clear entry point, offering an overview of the task structure.

Each entity within the data model is represented by its own panel (Fig. 4-2). Users can navigate to these panels to focus on specific aspects of a task by clicking on the ↗ icon next to the entity name (Fig. 4a). Entity Panels can also be opened within another Entity Panel when there are references between entities (Fig. 4b).

Through our use of Jelly, we recognized that some entities can be deeply nested within others. For example, `Dietary Restrictions` is not originally displayed in the home panel. Therefore, retrieving all `Dietary Restrictions` requires navigating through the `Dish` entity panel first, making accessing and editing these entities cumbersome. To address this, Jelly allows users to view all entities and



a Map View (default) b List View c Table View

Figure 5: Jelly currently supports switching between three different representations of data for an entity pane, including (a) *Map View*, (b) *List View* (where each item is expanded to show more details, e.g., the Shopping Items attribute is directly rendered for people to directly scroll through the list), and (c) Table View (where attributes may be folded to fit in the cells, e.g., the Shopping Items list is folded into a button. The user may click on it to reveal the full list).

open their respective panels directly using the ⊕ button (Fig. 4c), reducing unnecessary navigation steps.

Additionally, panels can be closed, resized, or rearranged, allowing users to customize their workspaces as needed.

6.1.2 Organizing Collections of Objects within Panels. Given the nested structure of the object-relational schema in our underlying data model, it is important to effectively show collections of objects in the panels. Our design goal is to support users in efficiently navigating complex data structures while maintaining an overview of key information. As described in Section 5.2.1, we support two rendering types to achieve this:

Expanded rendering presents a full list of items, with each item displaying a subset of attributes most relevant to the task. For instance, in the `Dinner Plan` panel, the `Menu` attribute is an array of `Dish`, displaying as a list of items on the interface, with each item only showing its name and cuisine. Clicking on an item opens a popup card, showing full detailed information, such as ingredients and dietary suitability of the dish (Fig. 4d). We choose the popup as a default form of revealing the details for in-situ inspection. Alternatively, users can choose to convert the popup card into a persistent floating card with 🗔 for easy reference; or open it in a dedicated entity panel for a more focused view.

Summary rendering condenses the collection into a single summary button, showing only the most relevant information for the task at hand. For example, in a `Shopping Plan`, a list of shopping items can be represented as a button showing the total number of `Shopping Items`, which users can click to reveal and expand into a full list (Fig. 5c). Similarly, within a `Travel Plan`, the `Budget` may be summarized as the total sum of all expenses, with an option to click and reveal a detailed breakdown.

6.1.3 Cross-referencing with Synchronized Highlighting. As mentioned above, an entity object can have multiple distinct representations within the interface. For example, an `Ingredient` object may appear in the home panel, and as an item in both the `Store` and `Menu` panel (Fig. 4e). To facilitate cross-referencing across different views, Jelly implements synchronized highlighting. When users hover over one object, all other objects containing the same instance are highlighted simultaneously. This helps users quickly identify related information across different contexts.

6.2 Interaction Techniques for Customization

Jelly allows users to customize the generated UIs with both natural language and direct manipulation, accommodating different types of customization needs.

6.2.1 Continuously Prompting with Traceable History. Users can give follow-up prompts in the chat view to continuously update the data model and the rendered UIs (Section 5.3). Each prompt also serves as an interactive history entry, as it preserves the state of the data model and UI specifications. Users can easily revisit any previous workspaces by clicking on corresponding messages (Fig. 4f). Additionally, any user customization made through the GUI, as described in the following section, is translated into an action-tagged entry. With the traceable history, users can easily switch between different versions of the interface geared towards specific tasks, or revert any changes if there are adjustments that do not meet their expectations.

6.2.2 GUI-Based Data Model Customization. While continuous prompting enables users to issue high-level requests and make complex structural changes, Jelly also provides GUI-based direct manipulation for more granular customization of both data and schema elements.

Data Customization. Data in Jelly are editable with suitable representations (Fig. 4g). Additionally, the object-oriented underlying structure enables encapsulated actions on entity instances. Users can generate additional instances of an entity (e.g., adding more dishes to a menu) by clicking the ⊞ button. Alternatively, they can add empty instances by clicking ⬚ (Fig. 6a). This allows them to fill in the values manually. In many cases, users may only know partial attributes of an entity. For example, Millie wants to add `Carbonara` as a dish for the dinner party, but she does not know the ingredients of it. To support this common need, Jelly provides an auto-complete feature: clicking the 🪄 button allows the user to automatically fill in missing attributes, which also triggers a prompt box at the top of the card, allowing users to specify preferences for the generated attributes (Fig. 6b–c).

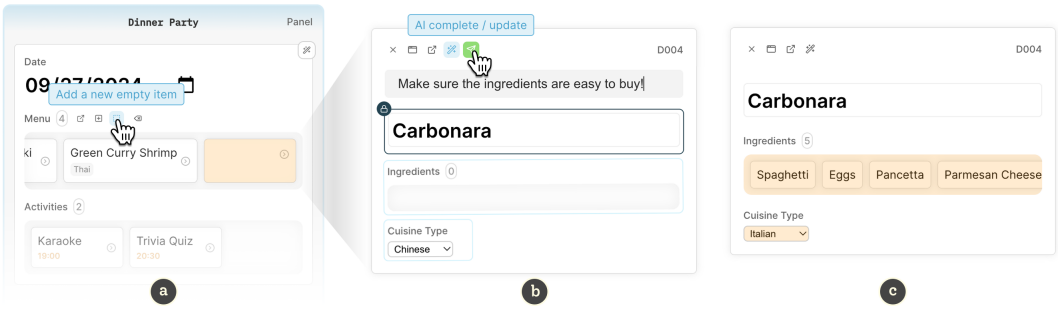


Figure 6: Generating Structured Data with Jelly. Users can create an empty object card (a), manually input some of the attributes, and optionally provide a prompt specifying their preference over other attributes (b). Jelly then auto-completes the remaining attributes based on the provided input (c).

Schema Customization. Beyond data customization, Jelly allows users to directly delete unnecessary attributes using ⌫ button next to the attribute name (Fig. 4h). However, adding or modifying attributes currently requires using the continuous prompting in the chat view. The implemented interactions represent only a subset of the possible customization techniques that could be integrated into Jelly. Given the proposed data model and UI specifications, we anticipate multiple interaction techniques for malleable UIs can be applied here. For example, users could customize which attributes to display in an expanded list when rendering a collection of objects (Section 6.1.2), as explored in recent research on malleable overview-detail interfaces [35].

6.2.3 Switching between Representations of Data. Even when using the same schema and data, users may prefer different representations depending on specific tasks. For example, a *list* facilitates browsing, such as viewing a set of places to visit; a *map* visualizes spatial relationships for route planning; a *table* makes it easier to compare attributes across items for decision makings.

To accommodate this, Jelly allows users to flexibly switch between representations within an entity panel, which displays multiple instances of an entity. Jelly automatically selects the most suitable representation based on the task, and provides a dropdown menu at the top-right of the panel for users to switch at any time (Fig. 5). While the current implementation only supports list, table, and map views, Jelly's infrastructure allows for easy extension to include other representations, such as timelines, stacks, or even user-defined representations tailored to specific needs.

7 Technical Evaluation

To evaluate the pipeline's ability to generate the task-driven data model based on user requests, we conducted a technical evaluation to assess the quality of the object-relational schema generated by LLMs, in our case, the GPT-4o model, which we used in our pipeline for corresponding modules. Specifically, we aim to assess:

- **Schema Relevance**: whether the entities and attributes generated by the system are aligned with the user's task goals and contribute meaningfully to the task's completion.
- **Dependency Accuracy**: (a) the correctness of the relationships between entities, particularly whether the dependencies recognized by the system accurately model task-specific relationships; and (b) whether the corresponding mechanisms (i.e., update or validation) are correct.

We did not evaluate the UI-specific aspects (e.g., UI components rendering and view composition mechanisms) in this technical evaluation, as these aspects depend heavily on user interaction and will be evaluated in the subsequent user study (Section 8).

7.1 Setup

7.1.1 Dataset. We employed GPT-4o to generate a set of informational tasks that users might typically require interfaces to complete. The tasks spanned various domains to reflect the generalizability of the system across different informational needs. In total, our dataset comprises 25 task scenarios. To better understand the pipeline's ability to respond to prompts of different levels of detail, for each task, we generated two versions of task prompts—one less detailed and one more detailed, for example:

- Less Detailed Prompt: "*I want to plan a trip to Tokyo with my friends.*"
- More Detailed Prompt: "*I want to plan a 7-day trip to Tokyo with my friends, for food and cultural experiences.*"

This resulted in a dataset of 50 task requests, yielding a total of 197 entities, 1052 attributes, and 232 dependencies in all 50 corresponding data models[1].

7.1.2 Coding Process. Two coders familiar with database schema and JavaScript programming (for analyzing dependencies) were involved in the evaluation. Each coder inspected the generated data models using the deployed data models and recorded their assessments on a coding sheet. For each task, the coders performed the following assessments:

- **Schema Relevance:** Coders rated the relevance of each entity and attribute on a 4-point scale, ranging from (1) unreasonable (not relevant or redundant), (2) could be useful, (3) necessary and expected, to (4) useful and surprising.
- **Dependency Accuracy:** Coders assessed the dependency relationship among attributes and coded each as **C**orrect, **W**rong (e.g., missing or targeting incorrect attributes), or **R**edundant.

[1]The data models used for evaluation are available on an interactive website: https://jelly-modeleval.netlify.app/

Table 1: Entity and Attribute Coding Results, rated on a 4-point scale, 1 - not relevant or redundant, 2 - could be useful, 3 - necessary and expected, 4 - useful and surprising

		Less Detailed Prompts				More Detailed Prompts			
Entity	**Total**	102				95			
	Rating	**1**	**2**	**3**	**4**	**1**	**2**	**3**	**4**
	C1	0%	3.92%	96.08%	0%	0%	4.21%	95.79%	0%
	C2	0%	2.94%	92.16%	4.90%	3.16%	2.11%	93.68%	1.05%
	Mean	**0%**	**3.43%**	**94.12%**	**2.45%**	**1.58%**	**3.16%**	**94.74%**	**0.53%**
Attribute	**Total**	534				518			
	Rating	**1**	**2**	**3**	**4**	**1**	**2**	**3**	**4**
	C1	0.37%	5.81%	93.63%	0.19%	0%	0.97%	94.59%	0.97%
	C2	0%	2.62%	94.19%	3.18%	0.97%	1.93%	95.75%	0.77%
	Mean	**0.19%**	**4.22%**	**93.91%**	**1.69%**	**0.49%**	**1.45%**	**95.17%**	**0.87%**

Table 2: Dependency Coding Results

	Less Detail Prompts			More Detail Prompts		
Total	120			112		
Rating	Correct **(C)**	Wrong **(W)**	Redundant **(R)**	Correct **(C)**	Wrong **(W)**	Redundant **(R)**
Relationship	89.17%	5.83%	5.00%	93.75%	1.79%	4.46%
Mechanism	98.33%	1.67%	n/a	95.54%	4.46%	n/a

Additionally, coders analyzed whether each dependency included correct validation or update relationship mechanism as described in Section 5.1.2 (i.e., correct JavaScript expression or natural language description) and coded each as either **C**orrect or **W**rong.

For any discrepancies between the codes of the dependency, a third coder examined the case to resolve the discrepancy. We report and discuss the final coding results below.

7.2 Results

As shown in Table 1, our coding results indicate that the majority of entities (94.12% and 94.74% for less and more detailed prompts) and attributes (93.91% and 95.17% for less and more detailed prompts) inferred by LLMs are necessary and expected. These results demonstrate that the object-relational schema employed in our pipeline can effectively model users' tasks, providing relevant information tailored to their needs. In most cases, LLMs successfully generate meaningful entities and attributes. However, a common issue arises when LLMs interpret prompts too literally, leading to redundant attributes. For instance, in the task "*I want to buy a standing desk*," the system might generate an unnecessary "*purchase decision*" field. However, this only comprises less than 0.5% of the cases.

Results of the dependency labeling in Table 2 show an average accuracy of 91.5% for relationship and 96.9% for mechanism. We identified two common errors: *reversed relationships*, where the relationships reverse the source and target, and *redundant dependencies*, where the dependencies describe relationships that are already declared in the schema with the referencing attributes. Although these dependency issues do not significantly impact the overall effectiveness of the pipeline and can often be identified through the GUI or corrected with improved prompting or validation, they highlight areas for improvement in how dependencies are inferred and generated. When using LLMs to establish directional relationships, validation is necessary to ensure accuracy, which we have integrated into our pipeline.

8 User Study

We conducted a user study to gain a comprehensive understanding of the pipeline's capability in responding to real-world user requests, the effectiveness of the generated UIs, and the novel workflows and limitations that may emerge from the study. Specifically, we aim to answer the following research questions:

- **RQ1** Do the generated UIs effectively present information in a way that helps users accomplish their tasks?
- **RQ2** How easily and effectively can users customize and modify the UIs to adapt to their evolving tasks and needs?
- **RQ3** What is the user experience of interacting with a generative and malleable information space, and how does it compare to existing interfaces such as chat-based systems and traditional GUI-based apps?
- **RQ4** What challenges do users encounter when generating and adapting the UIs to suit their needs? Are these challenges primarily due to the limitations of the generation pipeline or the interaction mechanisms?

8.1 Participants

We recruited 8 participants (5 female and 3 male, aged 21–28) through the internal communication channels within a large public university. Participants, including students and research scientists, reported that they use diverse information systems in their daily life and work. All of them use generative AI tools (e.g., ChatGPT) on a daily basis.

8.2 Study Procedure

The study lasted approximately 80 minutes per participant. Seven sessions were conducted in person and one remotely via Zoom. For all sessions, the experimenter provided verbal instructions, and the participants interacted with the system by following the experimenter's instructions. All sessions were screen- and voice-recorded. Participants were provided with a consent form, which they reviewed and signed before the study. Participants received a 30 USD Amazon gift card for their participation. The study was divided into the following phases:

Introduction (5 minutes). Participants were first given a brief introduction to the research and an overview of the system, including the different parts of the interface and their functionality.

System Walk-through/Tutorial (15 minutes). Participants were guided through an example task—hosting a dinner party, where they interacted with the system to request information, modify the UIs, and explore customization options. Participants performed the interactions themselves while the experimenter provided guidance, helping them familiarize themselves with the system's capabilities before moving on to freeform tasks.

Freeform Tasks (2 tasks, each 20 minutes). Participants were asked to use the system to complete two tasks of their choice. During each task, participants were encouraged to make various requests, evaluate the interface generated by the system, and reflect on whether the interface met their needs. They were instructed to create at least two follow-up queries per task to assess how well the system supported UI modifications.

Questionnaire and Interview (20 minutes). After completing the tasks, participants filled out a 5-point Likert scale questionnaire evaluating different dimensions of the system and their overall experience. A semi-structured interview was also conducted to gather in-depth qualitative feedback.

8.3 Results

We summarize the results of the questionnaire, interview, and user behavior analysis for the free-form tasks.

8.3.1 Questionnaire Results. Our questionnaire targeted assessing the utility and effectiveness of the generated information, the layout of the interface, and the interactions with Jelly through prompting and direct manipulation (RQ1, RQ2). The results show that participants generally found the information presented on the interface relevant (6 strongly agree, 2 agree) and can help them achieve their tasks efficiently (2 strongly agree, 6 agree). Being able to express their personal needs and customize the interface accordingly is found to be easy (4 strongly agree, 4 agree) and useful (6 strongly agree, 2 agree). The panel layout and organization of information were found to be intuitive (6 strongly agree, 2 agree) and effective for information consumption (5 strongly agree, 2 agree, 1 neutral). Full questionnaire results can be found in Appendix B.

8.3.2 Open-Ended Tasks and Interaction Behaviors. To better understand how users achieve their freeform tasks with the task-driven, model-based UIs (RQ3, RQ4), we logged and analyzed participants' follow-up chat messages with Jelly for continuous customization of the interface (Fig. 7). We analyzed 14 out of 16 tasks (2 tasks missing due to the loss of P1's data), which included 120 follow-up messages. Occasionally, a single chat message contained multiple prompts with distinct requirements or pieces of information (see the example in Fig. 7). We separated these prompts from the chat messages, which resulted in a total of 131 prompts. We include the full set of initial and follow-up prompts issued by participants to Jelly in the supplementary materials.

We categorized the prompts participants initiated into *Learning Task* and *Planning Task*. The follow-up prompts are classified based on the intended updates of the data model—either the data or the schema—and the level of specificity. We visualize the logged results in Fig. 7, with details reported below.

Prompt Specificity. We expect Jelly to handle user prompts across varying levels of specificity. If a prompt specifies both *target* and *action* of the expected schema or data change (Section 5.3), e.g., "*Add weather to the homepage*" (P8), it is coded as *fully specified*. If either or both aspects are missing, a prompt is considered *underspecified*, e.g., "*Give me weather information*" (P6). Sometimes, participants may broadly ask for strategies, e.g., "*What should I write about in my personal statement?*" (P5); or simply provide contextual information, such as "*This is a solo trip by the way*" (P4). These prompts do not provide any indication of how the UI should change. We code these prompts as *unspecified* prompts.

Participants made a total of 91 *fully specified* prompts (69%), 27 *underspecified* prompts (21%), and 13 *unspecified* prompts (10%). Our results show that when participants prompt the system to adapt to their needs, they tend to be more specific on planning tasks (82% *fully specified*, 8% *underspecified*, 10% *unspecified*) than learning tasks (35% *fully specified*, 55% *underspecified*, 10% *unspecified*).

Modification Patterns. Participants issued a total of 118 *fully specified* or *underspecified* prompts to Jelly to express the desired changes to either the schema or the data to adjust the UIs to their needs. Among these, 86 were schema changes (57 adds, 10 removes, 19 updates), and 32 were data changes (19 adds, 1 remove, 12 updates), indicating participants mostly sought to expand the scope of information or request additional data throughout the tasks.

Our results also show different modification patterns on the two types of tasks: during learning tasks, participants primarily engaged with schema modifications (92% schema changes, 8% data changes), whereas in planning tasks, the need for data modifications was significantly higher (65% schema changes, 35% data changes).

Failure Cases. While Jelly was able to effectively interpret most user prompts and make corresponding schema and data changes, it occasionally failed during the study (a total of 3 times among 120 follow-up messages). We herein report these 3 failure cases:

Case 1: Jelly was unable to handle P3's request to customize the visual style of the UIs: "*Can you make the text size smaller?*" As a result, no schema or data change was made.

Case 2: P4 requested more acivities of different types to be displayed in separate rows, expecting Jelly to create a set of lists, each containing a type of activity. Instead, Jelly added a "*type*" attribute to all existing activities, failing to meet the intended outcome.

Case 3: After observing the error in Case 2, P4 repeated the same request. However, Jelly maintained the same UI without implementing the desired changes.

These failure cases pointed to the limitations of the comprehensiveness of Jelly's UI specifications, which we will discuss further in detail in Section 8.4.4.

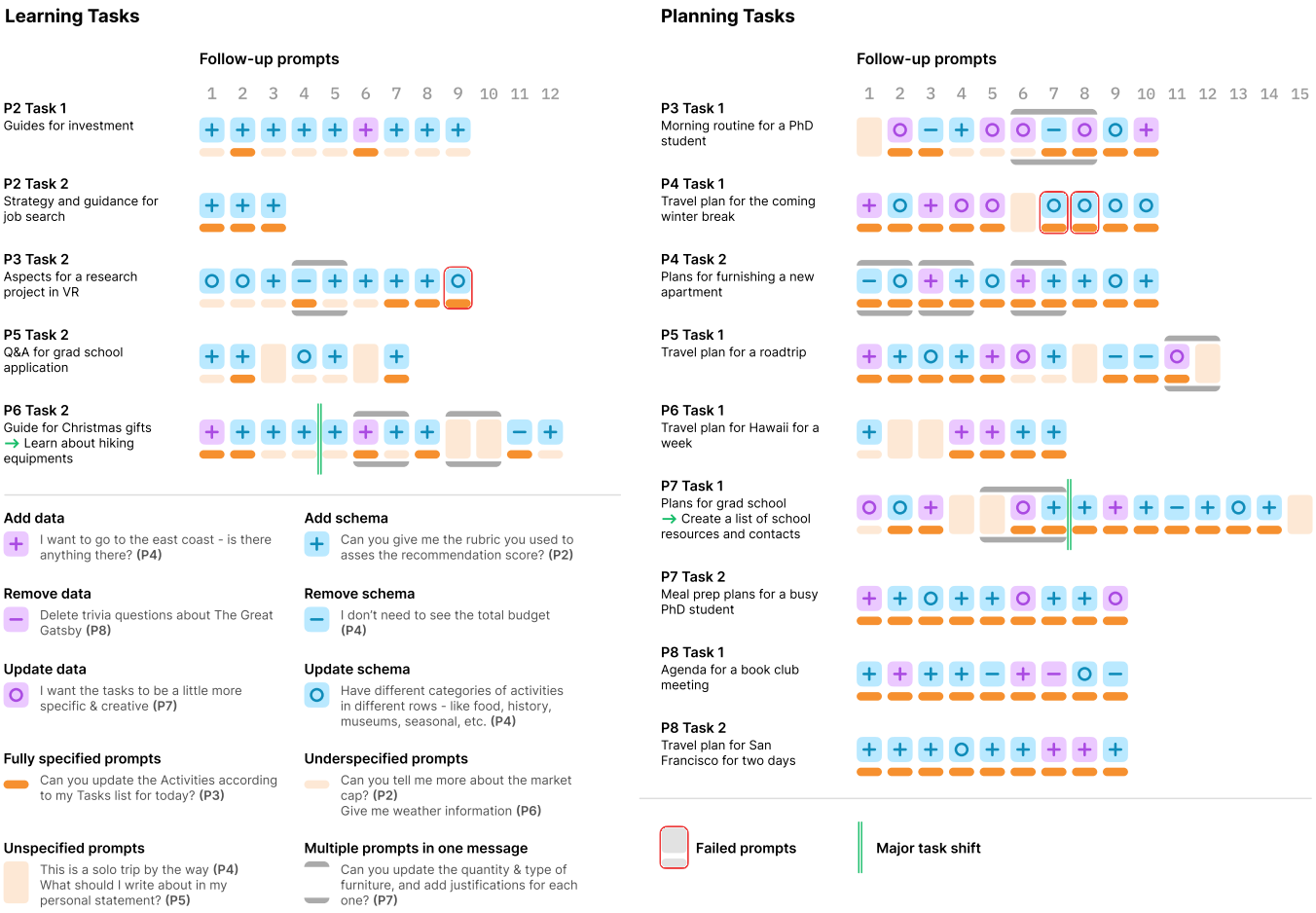


Figure 7: User study tasks and user prompts for continuous customization of the schema and data, organized by types of free-form tasks users chose during the study, including sensemaking and brainstorming tasks (left) and planning and logistical tasks (right). Types of coding and examples for each type are included below the logs.

8.4 Findings

All participants expressed excitement about being able to generate an information space tailored to their needs. The iterative customization experience was perceived "*fun*" (P2, P7) and "*efficient*" (P2, P4), allowing them to have an interface that "*reflects their needs along the way*" (P2). We discuss key findings and takeaways below.

8.4.1 Effective Information Organization for Task Achievement [RQ1]. Our results, as discussed in Section 8.3.1, revealed participants found the structured information in Jelly helped them effectively achieve their tasks. Participants also noted that they appreciated the system's ability to extend beyond their direct prompts and generate "*reasonable surprises*" that supported their goals (P4, P6). For example, when P4 requested new furniture for their living room planning task, Jelly went beyond the prompt and grouped furniture based on aesthetic themes and suggested corresponding vendors. They were fond of the unexpected grouping of furniture into "*Bohemian*" and "*Earth-toned*" categories, noting how it mirrored their aesthetic preferences and saved their efforts in creating such groupings manually. Moreover, Jelly commonly leveraged semantic linkages among those attributes with the proposed pipeline to streamline certain workflows. For example, when P7 requested dietary restrictions for all guests, Jelly not only applied those restrictions but also removed dishes that violated them, effectively anticipating the user's needs.

Takeaway: The structured organization and semantic associations enabled by the object-relational schema effectively present and manage LLM's open-world knowledge to be easier to consume and control by the end-users.

8.4.2 Continuous Customizability and Flexible Adaptation to User Needs and Tasks [RQ2]. One of the standout features of Jelly was its ability to accommodate continuous, iterative customization. P6 noted that many other tools offer "*one-shot*" customization, where users make changes that are meant to be permanent. Jelly's continuous adaptability allowed the users to adjust their workflows dynamically without being locked into a specific configuration. For example, P8 noted that they often struggled to decide which applications to use and had to manually collect information from multiple sources into different note-taking tools. With Jelly, they

could bypass the overhead of debating and selecting suitable applications as well as juggling multiple applications. They felt confident knowing they could always request Jelly to provide the desired information as new needs arose.

We observed participants naturally shifted focuses or changed task scopes when performing tasks with Jelly. For example, P6's task transformed from preparing Christmas gifts to learning and planning for a hiking trip, and P7 began with planning for settling into graduate school, which eventually scoped down to creating a list of school resources and managing their contacts. This fluidity was enabled by Jelly 's continuous customization, allowing participants to transition between evolving information needs without disrupting their current workflows.

Jelly's ability to interpret ambiguous prompts and leverage contextual information allowed participants to comfortably begin with vague inquiries and refine their goals as they interacted with the system. This was particularly beneficial when exploring unfamiliar domains. For example, during a trip to Hawaii, P6 stated, "*I want to stay on the beach*," without a clear idea of how this preference would impact their plans. They simply made the request out of curiosity to see how Jelly would handle it. In response, Jelly generated a list of beachfront hotels along with suggested beach activities, which inspired them to explore and plan different activities for the trip.

Participants also commented on the traceable history in the chat view, which allowed them to easily revisit previous states, especially when AI-generated changes did not fully align with their needs.

Takeaways: Users' intentions naturally shift and evolve during information tasks, varying in specificity. A system's ability to accommodate the varying levels of specificity and continuously adapt is essential for supporting fluid information activities. By leveraging LLMs' capacity to interpret flexible inputs and task-driven, model-based UI generation, and efficiently accomplish tasks.

8.4.3 Task-Driven Data Model: Malleable but Persistent Structures [RQ3]. Compared to chat-based systems and traditional apps, Jelly was seen as providing a novel, more fluid experience that combined the best of both worlds. The persistence and flexibility of the data model was considered beneficial in maintaining continuity with evolving tasks and able to offer desired interactions with the information they needed for the tasks (8 strongly agree).

Unlike existing chat-based interfaces with LLMs (e.g., ChatGPT), where responses primarily generate content, P6 noted that Jelly was "*generating the way to organize information.*" Participants also appreciated being able to make localized adjustments to a particular part of the interface—such as adding or editing a field—without needing to issue a full reset or restructuring of the entire layout, which is often the case in existing LLM-based generative systems (P2, P4). Moreover, the structure enabled users to modify and carry data seamlessly across ongoing tasks, reducing the need for copy-pasting and manual adjustments typically required in chat-based systems. Transparency was another key advantage of the model-driven approach, giving participants confidence that their task structures remained intact rather than being unpredictably altered by AI. As participants became more familiar with Jelly, they actively engaged with the schema view to explore and understand AI's modifications. For example, when P6 requested daily weather information for their itinerary, they first checked the schema view to confirm that the itinerary schema had been extended before switching back to the UI view to verify the changes.

Compared to existing apps, participants appreciated the absence of "*opinionated*" design choices that restrict customization (6 strongly agree, 1 agree, 1 neutral). P4 described situations where they could often achieve 80% of their desired functionality in existing apps, but the inability to make small, yet necessary, modifications for specific use cases was frustrating—such as splitting road trip costs in a collaborative travel planning application. P6 further elaborated that existing apps "*force everyone to see all the information*," whereas Jelly allowed users to view only what was relevant to them. This ability to "*own my data*" and structure it according to personal preferences was seen as one of the major strengths (P6).

Takeaways: Continuity and transparency are essential for a generated interface to effectively meet users' task needs. Anchoring the generation process to the flexible, task-driven data model ensures these qualities, enabling users to adapt structures to their preferences and maintain confidence in the UI modifications.

8.4.4 Expecting More Efficient Ways to Interact with the Model and UIs. One challenge of the current implementation of Jelly was the need for continuous prompting to make incremental changes for most of the cases. While all participants acknowledged that it was easy to articulate intended changes to AI (4 strongly agree, 4 agree), P1 and P8 noted that it could be tedious to describe every requirement in detail when the system failed to initially generate a sufficient task structure. A potential improvement is to generate the model along with possible expansions to it (e.g., additional entities and attributes to consider), which would enable users to quickly expand the model with lightweight interaction.

Three participants (P3, P6, and P7) found the schema view useful for inspecting the underlying structure and understanding the changes, while others mainly relied on the generated UI for their tasks. P4 suggested that enhancing interactions for manipulating the schema directly could be particularly beneficial for developers. For example, P4 expressed a desire to "*cherry-pick*" elements from one schema as the starting point for another session. This also points to a potential future direction for making schema composable and reusable to enable users to create new spaces from existing ones, facilitating cross-domain tasks.

Additionally, the generated UIs in Jelly incorporated a limited set of design patterns within its specifications, restricting the expressiveness of information presentation and interactions. For example, P2 expressed the desire to have a line chart visualization of stock information, which was beyond the scope of our system's current UI specifications. This limitation highlights the need for a more comprehensive UI specification to enhance information representation and interaction (e.g., diverse layouts, advanced interaction logic), which we discuss further in the following section.

Takeaways: Efficiency of information acquisition and expressiveness of UIs are desired by the users. While continuous prompting offers flexibility, more proactive model expansion and lightweight refinement mechanisms are needed for achieving composable and reusable UIs to further adapt to evolving task needs. Enhancing UI design patterns and expanding UI specifications would further improve information expressiveness and usability.

9 Discussion and Future Directions on Generative and Malleable UI

In this section, we reflect on our findings, their implications, and future directions for advancing generative and malleable UIs. We revisit the five key aspects outlined in Section 2.1, structuring our discussion around them.

9.1 Broadening Supported Tasks with Advanced Dependency Modeling

Our technical pipeline currently models the dependencies required in an information task by describing the relationships among pairs of source and target elements. While this relatively simple mechanism yields few errors in our technical evaluation and is sufficient in supporting study participants in completing their intended tasks, it also has limitations in describing tasks that require complex interaction. A future direction is to explore more advanced graph-based dependency modeling, where nodes of the graph represent entities and attributes, and the edges represent the dependency relationships expressed using a more expressive specification language. This dependency graph will not only allow for the modeling of interaction logic beyond pairs of elements but also will enable end-users to intuitively inspect the dependencies by leveraging representation and interaction techniques introduced in graph-based visual programming.

9.2 Supporting Information Transformation Patterns with Higher-level Schema Operations

While the current schema operations (e.g., add, delete, and update) could theoretically handle all possible schema transformations, having to translate high-level transformations to these low-level operations can lead to complex and error-prone data and UI modifications—especially when LLMs are involved in the process. High-level transformations, such as *eversion*, are commonly needed but difficult to express with atomic operations. For example, a user might start by viewing a list of literature modeled as an `Publication` entity containing attributes like title, authors, and year. Later, they might wish to transform this view to focus on all `Authors` and their respective publications. Using current schema operations, the system would need to generate a new entity, likely resulting in substantial changes to both the UI and underlying data. A dedicated *eversion* schema operation from the `Publication` entity to the `Author` entity could ensure a smooth data and UI transformation. A future direction is to analyze users' prompts with the system to identify the desired high-level transformations and expand the schema operations to support them more directly.

9.3 Enabling Advanced and Malleable View Management

The current Jelly implementation employs a column-based layout to organize the panels on the screen. We intentionally opted for this layout as it avoids the tedious and manual positioning of the panels while ensuring sufficient usability of the generated UIs. Its simplicity allows us to focus our research on investigating the underlying technical pipeline. View and layout management in itself is a long-standing research theme in HCI [9, 20, 36, 71]. Among the various methods, a straightforward extension of the Jelly system is to integrate the underlying data model with the dashboard design pattern, which summarizes the key dimensions that guide the placement and presentation of information panels [9]. Concretely, entities and lists of entities can be displayed in separate panels, arranged based on inferred importance—such as the number of attributes or connections to other entities. Key panels could occupy central positions with detailed information, whereas less critical ones might be placed along the periphery with more condensed views. Certain entities may benefit from being displayed with multiple synchronized representations (e.g., a table and a map). By supporting the diverse representations of information within each panel, we can further enhance Jelly's expressivity in the graphical representation of information.

9.4 Integrating with External and User Data and Enhanced Data Transformation and Management

An immediate next step is to extend our technical pipeline with external data sources and user-permitted data beyond just relying on LLM-generated data. Recent approaches of integrating LLMs with external data—such as Retrieval-Augmented Generation [28], the Model-Context Protocol [5], and LLM-generated API calls [62]—offer promising ways of integrating reliable data sources into our technical pipeline. In addition to connecting with the various data sources, it is also worth systematically exploring the underlying data infrastructure with regard to how data is managed and transformed. The benefit of a persistent schema employed by traditional applications is that it may be optimized based on data storage and retrieval efficiency. The dynamic data schema that drives the interface may not directly mapped to the underlying database schema. Our future work will investigate a data transformation layer to facilitate seamless acquisition and data adaptation in response to evolving schema changes.

9.5 Enabling Personalized UIs with Context Preservation

Personalized interfaces have been a long-standing endeavor in activity-centered computing [3, 23], which require not only customization but also context preservation. Unlike most existing interfaces that remain the same regardless of who uses them and for how long, we will explore how the underlying data model and the generated UIs can become personalized and intelligently tailored to each individual context and preference over time. A key challenge is balancing adaptability and predictability. An interface that adapts too aggressively may make incorrect assumptions about user intent, leading to frustration; conversely, an interface that requires excessive manual configurations, shifts too much burden on the user. Our insight is that achieving context-aware UIs requires an intermediate representation that effectively preserves and reuses user context and hence guides adaptation.

The data model presents a promising solution. Jelly can record users' preferred entities, attributes, and interface configurations for different tasks. When a user encounters new tasks, it can intelligently reuse subcomponents from previous relevant tasks. For example, if a user is organizing an academic workshop for the first

time, the system could adapt UI elements from workspaces of past activities, such as scheduling talks (as in conference planning) or coordinating a dinner event (as in family gathering). Besides model reuse, future work will also explore personalized model evolution. Different users may prioritize different aspects of their workflows—a researcher may focus on entity relationships when conducting a literature review, while a project manager may emphasize task dependencies and deadlines. Through the interactions we explore with the data model, users should be able to inspect and customize the context being preserved and adapted, achieving an information space that is not only generative, malleable, but also personal.

10 Conclusion

As users can dynamically express arbitrary prompts to AI describing their intended information activity, a demand arises for generative and malleable user interfaces. This poses an enormous and exciting challenge for the HCI community—how can we design interfaces that can support information activities of any domain, scope, and complexity? This work provides an exploration towards this goal. Taking the perspective that a GUI-based interactive system is the graphical representation of the data model that describes the targeted user tasks, we recognize that generative and malleable user interfaces fundamentally demand generative and malleable data models to support users' dynamic tasks. Therefore, we propose leveraging LLMs to generate task-driven data models based on the tasks indicated in users' prompts, which then guide the generation of the user interface. Results from our technical evaluation show that LLMs can generate relatively high quality data models. We implemented the proposed generation pipeline into a prototype system Jelly. The user evaluation of the system shows that the generative and malleable user interfaces enable users to develop a personalized and dynamic information space by flexibly curating diverse information and customizing its representation. Our research demonstrates a promising approach and points out a wide range of exciting future research directions to realize the long-envisioned generative, malleable user interfaces.

References

[1] Gregory D Abowd, Anind K Dey, Peter J Brown, Nigel Davies, Mark Smith, and Pete Steggles. 1999. Towards a better understanding of context and context-awareness. In *Handheld and Ubiquitous Computing: First International Symposium, HUC'99 Karlsruhe, Germany, September 27–29, 1999 Proceedings 1*. Springer, 304–307.

[2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[3] Eytan Adar, David Karger, and Lynn Andrea Stein. 1999. Haystack: Per-user information environments. In *Proceedings of the eighth international conference on Information and knowledge management*. 413–422.

[4] Anthropic. 2024. Claude-3.5. https://www.anthropic.com.

[5] Anthropic. n.d.. Model Context Protocol. https://www.anthropic.com/news/model-context-protocol Retrieved December 9, 2024.

[6] Apple Inc. 1987. Knowledge Navigator. https://en.wikipedia.org/wiki/Knowledge_Navigator Concept video and vision for future technology.

[7] Apple Inc. 1995. *OpenDoc.* https://en.wikipedia.org/wiki/OpenDoc Software framework for compound documents.

[8] Bill Atkinson. 1987. *Hypercard.* https://en.wikipedia.org/wiki/HyperCard

[9] Benjamin Bach, Euan Freeman, Alfie Abdul-Rahman, Cagatay Turkay, Saiful Khan, Yulei Fan, and Min Chen. 2022. Dashboard design patterns. *IEEE Transactions on Visualization and Computer Graphics* 29, 1 (2022), 342–352.

[10] Yining Cao, Yiyi Huang, Anh Truong, Hijung Valentina Shin, and Haijun Xia. 2025. Compositional Structures as Substrates for Human-AI Co-creation Environment: A Design Approach and A Case Study. In *CHI Conference on Human Factors in Computing Systems (CHI '25)* (Yokohama, Japan). ACM, New York, NY, USA, 25. doi:10.1145/3706598.3713401

[11] Anind K Dey, Gregory D Abowd, and Daniel Salber. 2001. A conceptual framework and a toolkit for supporting the rapid prototyping of context-aware applications. *Human–Computer Interaction* 16, 2-4 (2001), 97–166.

[12] Morgan Dixon and James Fogarty. 2010. Prefab: implementing advanced behaviors using pixel-based reverse engineering of interface structure. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Atlanta, Georgia, USA) *(CHI '10)*. Association for Computing Machinery, New York, NY, USA, 1525–1534. doi:10.1145/1753326.1753554

[13] James Foley, Won Chul Kim, Srdjan Kovacevic, and Kevin Murray. 1991. UIDE—an intelligent user interface design environment. In *Intelligent user interfaces*. 339–384.

[14] Amy Rae Fox, Philip Guo, Clemens Nylandsted Klokmose, Peter Dalsgaard, Arvind Satyanarayan, Haijun Xia, and James D. Hollan. 2020. Towards a dynamic multiscale personal information space: beyond application and document centered views of information. In *Companion Proceedings of the 4th International Conference on Art, Science, and Engineering of Programming* (Porto, Portugal) *(Programming '20)*. Association for Computing Machinery, New York, NY, USA, 136–143. doi:10.1145/3397537.3397542

[15] Jun Fujima, Aran Lunzer, Kasper Hornbæk, and Yuzuru Tanaka. 2004. C3W: clipping, connecting and cloning for the web. In *Proceedings of the 13th International World Wide Web Conference on Alternate Track Papers & Posters* (New York, NY, USA) *(WWW Alt. '04)*. Association for Computing Machinery, New York, NY, USA, 444–445. doi:10.1145/1013367.1013517

[16] Krzysztof Gajos and Daniel S Weld. 2004. SUPPLE: automatically generating user interfaces. In *Proceedings of the 9th international conference on Intelligent user interfaces*. 93–100.

[17] Krzysztof Z Gajos, Daniel S Weld, and Jacob O Wobbrock. 2010. Automatically generating personalized user interfaces with Supple. *Artificial intelligence* 174, 12-13 (2010), 910–950.

[18] Krzysztof Z. Gajos, Jacob O. Wobbrock, and Daniel S. Weld. 2007. Automatically generating user interfaces adapted to users' motor and vision capabilities. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology* (Newport, Rhode Island, USA) *(UIST '07)*. Association for Computing Machinery, New York, NY, USA, 231–240. doi:10.1145/1294211.1294253

[19] Jeffrey Heer and Michael Bostock. 2010. Declarative language design for interactive visualization. *IEEE transactions on visualization and computer graphics* 16, 6 (2010), 1149–1156.

[20] Scott E. Hudson and Shamim P. Mohamed. 1990. Interactive specification of flexible user interface displays. *ACM Trans. Inf. Syst.* 8, 3 (July 1990), 269–288. doi:10.1145/98188.98201

[21] Robert J. K. Jacob. 1983. Using formal specifications in the design of a human-computer interface. *Commun. ACM* 26, 4 (April 1983), 259–264. doi:10.1145/2163.358093

[22] Yue Jiang, Yuwen Lu, Jeffrey Nichols, Wolfgang Stuerzlinger, Chun Yu, Christof Lutteroth, Yang Li, Ranjitha Kumar, and Toby Jia-Jun Li. 2022. Computational Approaches for Understanding, Generating, and Adapting User Interfaces. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) *(CHI EA '22)*. Association for Computing Machinery, New York, NY, USA, Article 74, 6 pages. doi:10.1145/3491101.3504030

[23] David Karger and Lynn Andrea Stein. 1997. Haystack: Per-User Information Environments. *White Paper* (1997).

[24] Alan C Kay. 1996. The early history of Smalltalk. In *History of programming languages—II*. 511–598.

[25] Scott Robert Klemmer. 2004. *Tangible user interface input: Tools and techniques*. University of California, Berkeley.

[26] Amy J Ko, Brad A Myers, and Htet Htet Aung. 2004. Six learning barriers in end-user programming systems. In *2004 IEEE Symposium on Visual Languages-Human Centric Computing*. IEEE, 199–206.

[27] Glenn E Krasner, Stephen T Pope, et al. 1988. A description of the model-view-controller user interface paradigm in the smalltalk-80 system. *Journal of object oriented programming* 1, 3 (1988), 26–49.

[28] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[29] Toby Jia-Jun Li, Marissa Radensky, Justin Jia, Kirielle Singarajah, Tom M. Mitchell, and Brad A. Myers. 2019. PUMICE: A Multi-Modal Agent that Learns Concepts and Conditionals from Natural Language and Demonstrations. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology* (New Orleans, LA, USA) *(UIST '19)*. Association for Computing Machinery, New York, NY, USA, 577–589. doi:10.1145/3332165.3347899

[30] James Lin, Jeffrey Wong, Jeffrey Nichols, Allen Cypher, and Tessa A. Lau. 2009. End-user programming of mashups with vegemite. In *Proceedings of the 14th International Conference on Intelligent User Interfaces* (Sanibel Island, Florida, USA) *(IUI '09)*. Association for Computing Machinery, New York, NY, USA, 97–106.

doi:10.1145/1502650.1502667
[31] David Lindlbauer, Anna Maria Feit, and Otmar Hilliges. 2019. Context-aware online adaptation of mixed reality interfaces. In *Proceedings of the 32nd annual ACM symposium on user interface software and technology*. 147–160.
[32] Geoffrey Litt and Daniel Jackson. 2020. Wildcard: spreadsheet-driven customization of web applications. In *Companion Proceedings of the 4th International Conference on Art, Science, and Engineering of Programming*. 126–135.
[33] Vivian Liu and Lydia B Chilton. 2022. Design Guidelines for Prompt Engineering Text-to-Image Generative Models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) *(CHI '22)*. Association for Computing Machinery, New York, NY, USA, Article 384, 23 pages. doi:10.1145/3491102.3501825
[34] Ping Luo, Pedro Szekely, and Robert Neches. 1993. Management of interface design in HUMANOID. In *Proceedings of the INTERACT'93 and CHI'93 Conference on Human Factors in Computing Systems*. 107–114.
[35] Bryan Min, Allen Chen, Yining Cao, and Haijun Xia. 2025. Malleable Overview-Detail Interfaces. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '25)*. Association for Computing Machinery, New York, NY, USA, 26 pages. doi:10.1145/3706598.3714164
[36] Brad Myers, Scott E. Hudson, and Randy Pausch. 2000. Past, present, and future of user interface software tools. *ACM Trans. Comput.-Hum. Interact.* 7, 1 (March 2000), 3–28. doi:10.1145/344949.344959
[37] Brad A. Myers. 1995. User interface software tools. *ACM Trans. Comput.-Hum. Interact.* 2, 1 (March 1995), 64–103. doi:10.1145/200968.200971
[38] Brad A Myers. 1995. User interface software tools. *ACM Transactions on Computer-Human Interaction (TOCHI)* 2, 1 (1995), 64–103.
[39] Brad A. Myers and Mary Beth Rosson. 1992. Survey on user interface programming. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Monterey, California, USA) *(CHI '92)*. Association for Computing Machinery, New York, NY, USA, 195–202. doi:10.1145/142750.142789
[40] Bonnie A Nardi. 1993. *A small matter of programming: perspectives on end user computing*. MIT press.
[41] Arpit Narechania, Arjun Srinivasan, and John Stasko. 2020. NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics* 27, 2 (2020), 369–379.
[42] Theodor Holm Nelson. 1995. The heart of connection: hypermedia unified by transclusion. *Commun. ACM* 38, 8 (aug 1995), 31–33. doi:10.1145/208344.208353
[43] Jeffrey Nichols, Brad A. Myers, Michael Higgins, Joseph Hughes, Thomas K. Harris, Roni Rosenfeld, and Mathilde Pignol. 2002. Generating remote control interfaces for complex appliances. In *Proceedings of the 15th Annual ACM Symposium on User Interface Software and Technology* (Paris, France) *(UIST '02)*. Association for Computing Machinery, New York, NY, USA, 161–170. doi:10.1145/571985.572008
[44] Jeffrey Nichols, Brad A. Myers, and Kevin Litwack. 2004. Improving automatic interface generation with smart templates. In *Proceedings of the 9th International Conference on Intelligent User Interfaces* (Funchal, Madeira, Portugal) *(IUI '04)*. Association for Computing Machinery, New York, NY, USA, 286–288. doi:10.1145/964442.964507
[45] Jeffrey Nichols, Brad A. Myers, and Brandon Rothrock. 2006. UNIFORM: automatically generating consistent remote control user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Montréal, Québec, Canada) *(CHI '06)*. Association for Computing Machinery, New York, NY, USA, 611–620. doi:10.1145/1124772.1124865
[46] Jeffrey Nichols, Brandon Rothrock, Duen Horng Chau, and Brad A. Myers. 2006. Huddle: automatically generating interfaces for systems of multiple connected appliances. In *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology* (Montreux, Switzerland) *(UIST '06)*. Association for Computing Machinery, New York, NY, USA, 279–288. doi:10.1145/1166253.1166298
[47] Dan Olsen. 1992. *User interface management systems: models and algorithms*. Morgan Kaufmann Publishers Inc.
[48] Dan R Olsen Jr. 1986. MIKE: The menu interaction kontrol environment. *ACM Transactions on Graphics (TOG)* 5, 4 (1986), 318–344.
[49] Jared Palmer. 2023. Announcing v0: Generative UI. https://vercel.com/blog/announcing-v0-generative-ui Accessed: 2024-08-28.
[50] Angel Puerta and Jacob Eisenstein. 1998. Towards a general computational framework for model-based interface development systems. In *Proceedings of the 4th international conference on Intelligent user interfaces*. 171–178.
[51] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. 2009. Scratch: Programming for All. *Communications of the ACM (CACM)* (2009).
[52] Advait Sarkar, Andrew D Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence? *arXiv preprint arXiv:2208.06213* (2022).
[53] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2016. Vega-lite: A grammar of interactive graphics. *IEEE transactions on visualization and computer graphics* 23, 1 (2016), 341–350.
[54] Semantic Scholar. n.d.. Semantic Scholar Academic Graph API. https://www.semanticscholar.org/product/api Accessed: February 12, 2025.
[55] Siegfried Schreiber. 1994. Specification and generation of user interfaces with the boss-system. In *International Conference on Human-Computer Interaction*. Springer, 107–120.
[56] Paul Sonnentag, Alexander Obenauer, and Geoffrey Litt. 2023. Embark: Dynamic Documents as Personal Software. In *LIVE 2023: The Ninth Workshop on Live Programming*. Cascais, PT. https://inkandswitch.com/embark
[57] Pedro Szekely. 1996. Retrospective and challenges for model-based interface development. In *Design, Specification and Verification of Interactive Systems' 96: Proceedings of the Eurographics Workshop in Namur, Belgium, June 5–7, 1996*. Springer, 1–27.
[58] Pedro Szekely, Ping Luo, and Robert Neches. 1992. Facilitating the exploration of interface design alternatives: The HUMANOID model of interface design. In *Proceedings of the SIGCHI conference on Human factors in computing systems*. 507–515.
[59] Pedro Szekely, Piyawadee Sukaviriya, Pablo Castells, Jeyakumar Muthukumarasamy, and Ewald Salcher. 1996. Declarative interface models for user interface construction tools: The M astermind approach. In *Engineering for Human-Computer Interaction: Proceedings of the IFIP TC2/WG2. 7 working conference on engineering for human-computer interaction, Yellowstone Park, USA, August 1995 6*. Springer, 120–150.
[60] Jenifer Tidwell. 1997. A pattern language for human-computer interface design. *Available via DIALOG* (1997).
[61] TripAdvisor. n.d.. TripAdvisor Content API. https://developer-tripadvisor.com/content-api/ Accessed: February 12, 2025.
[62] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. *arXiv preprint arXiv:2407.18901* (2024).
[63] Priyan Vaithilingam, Elena L Glassman, Jeevana Priya Inala, and Chenglong Wang. 2024. DynaVis: Dynamically Synthesized UI Widgets for Visualization Editing. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 1–17.
[64] Priyan Vaithilingam and Philip J. Guo. 2019. Bespoke: Interactively Synthesizing Custom GUIs from Command-Line Applications By Demonstration. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology* (New Orleans, LA, USA) *(UIST '19)*. Association for Computing Machinery, New York, NY, USA, 563–576. doi:10.1145/3332165.3347944
[65] Jean Vanderdonckt. 1995. Knowledge-based systems for automated user interface generation: the trident experience. In *Proceedings of the CHI*, Vol. 95.
[66] Bret Victor. 2020. *DynamicLand*. https://dynamicland.org/
[67] Charles Wiecha, William Bennett, Stephen Boies, John Gould, and Sharon Greene. 1990. ITS: A tool for rapidly developing interactive applications. *ACM Transactions on Information Systems (TOIS)* 8, 3 (1990), 204–236.
[68] Jacob O Wobbrock, Shaun K Kane, Krzysztof Z Gajos, Susumu Harada, and Jon Froehlich. 2011. Ability-based design: Concept, principles and examples. *ACM Transactions on Accessible Computing (TACCESS)* 3, 3 (2011), 1–27.
[69] Jason Wu, Eldon Schoop, Alan Leung, Titus Barik, Jeffrey P Bigham, and Jeffrey Nichols. 2024. UICoder: Finetuning Large Language Models to Generate User Interface Code through Automated Feedback. *arXiv:2406.07739* (2024).
[70] J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 437, 21 pages. doi:10.1145/3544548.3581388
[71] Clemens Zeidler, Christof Lutteroth, Wolfgang Sturzlinger, and Gerald Weber. 2013. The auckland layout editor: An improved gui layout specification process. In *Proceedings of the 26th annual ACM symposium on User interface software and technology*. 343–352.
[72] Amy X. Zhang, Michael Muller, and Dakuo Wang. 2020. How do Data Science Workers Collaborate? Roles, Workflows, and Tools. *Proc. ACM Hum.-Comput. Interact.* 4, CSCW1, Article 22 (may 2020), 23 pages. doi:10.1145/3392826
[73] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. 2023. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3836–3847.
[74] Xiaoyi Zhang, Lilian de Greef, Amanda Swearngin, Samuel White, Kyle Murray, Lisa Yu, Qi Shan, Jeffrey Nichols, Jason Wu, Chris Fleizach, Aaron Everitt, and Jeffrey P Bigham. 2021. Screen Recognition: Creating Accessibility Metadata for Mobile Applications from Pixels. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 275, 15 pages. doi:10.1145/3411764.3445186
[75] Xiong Zhang and Philip J. Guo. 2018. Fusion: Opportunistic Web Prototyping with UI Mashups. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology* (Berlin, Germany) *(UIST '18)*. Association for Computing Machinery, New York, NY, USA, 951–962. doi:10.1145/3242587.3242632

A UI Specification

Table 3: UI specification for one attribute of an object

Key	Value	Explanation
type	`string`	The data type of the value, which can be a string, number, array, or the name of an entity, for which we use `__<ENTITY>__` to annotate, e.g., `__PERSON__`
	`number`	
	`array`	
	`__<ENTITY>__`	
function	`priviateIdentifier`	The attribute functions as a unique identifier of an object used internally by the Jelly system. Private identifiers may not be semantically meaningful to the user and should not be displayed in the interface
	`publicIdentifier`	The attribute functions as a representative identity of an object, such as name and title. Public identifiers should be displayed with the highest saliency when rendering the UI for the object
	`display`	All other attributes of the object
editable	`true` or `false`	Whether the user is allowed to modify the value of the attribute in the rendered UI
		When the `type` of the attribute is not `array` ...
render	`shortText`	Short pieces of text, e.g., name of a hotel, title of a book
	`paragraph`	Long blocks of text, e.g., description of a city, review of a product
	`number`	Numeric values, including integers, floats, percentages, etc.
	`url`	Links to websites, e.g., https://hci.ucsd.edu
	`time`	Temporal values, such as dates, specific points in time, durations, etc.
	`location`	Geographic coordinates or names of places
	`category`	One of the categories defined by the `categories` field as a list of strings in the attribute specification
	`hidden`	The attribute will not be rendered
		When the `type` of the attribute is `array` ...
render	`summary`	The array is minimized as a single button showing one key aspect of the items in the array (see below for how it's derived). The full array shows upon clicked
	`expanded`	The array is fully shown, and the user can directly see, scroll through, and interact with each item
item	`type`	The type of the items in the array, same as `type` for the attributes
	`thumbnail`	An array of attribute names. If the item type is an entity, we need a set of representative and relevant attributes when displayed in a minimized form in the array (default to `publicIdentifier`)
(summary)	`{ name, derived }`	Only for the `summary` render type. The target attribute of the array items (`name`) and the method for deriving the summarizing value from them (`derived`). `derived` is an object of two keys, `field` and `operation`. If `field` is a number, `operation` could be `SUM`, `AVG`, `MIN`, or `MAX`; or `FILTER` or `COUNT` if `field` is an array

A.1 Example Specification

```
{
  DINNER_PLAN: {
    id: { type: "string", editable: true , render: "hidden", function: "privateIdentifier" },
    date: { type: "string", editable: true , render: "date", function: "display" },
    host: { type: "__USER__", editable: true , render: "shortText", function: "display" },
    location: { type: "string", editable: true , render: "location", function: "display" },
    guest_list: {
      type: "array",
      editable: true ,
      render: "expanded",
      item: { type: "__USER__", thumbnail: ["name", "phone"] }
    },
    menu: {
      type: "array",
      editable: true ,
      render: "summary",
      summary: {
        name: "total_calories",
        derived: { operation: "SUM", field: "calories" }
      },
      item: { type: "__DISH__", thumbnail: ["name", "calories"] }
    }
  },
  USER: {
    id: { type: "string", editable: true , render: "hidden", function: "privateIdentifier" },
    name: { type: "string", editable: true , render: "shortText", function: "publicIdentifier" },
    email: { type: "string", editable: true , render: "url", function: "display" },
    phone: { type: "string", editable: true , render: "number", function: "display" }
  },
  DISH: {
    id: { type: "string", editable: true , render: "hidden", function: "privateIdentifier" },
    name: { type: "string", editable: true , render: "shortText", function: "publicIdentifier" },
    ingredients: {
      type: "array",
      editable: true ,
      render: "expanded",
      item: { type: "string", editable: true , render: "shortText", function: "display" }
    },
    calories: { type: "number", editable: true , render: "number", function: "display" },
    cuisine_type: {
      type: "string",
      editable: true ,
      render: "category",
      function: "display",
      categories: ["American", "Italian", "Chinese", "Japanese", "French"]
    }
  }
}
```

B User Study Questionnaire and Responses

We plot the summary of all 5-point Likert scale questionnaire questions and participant responses here.

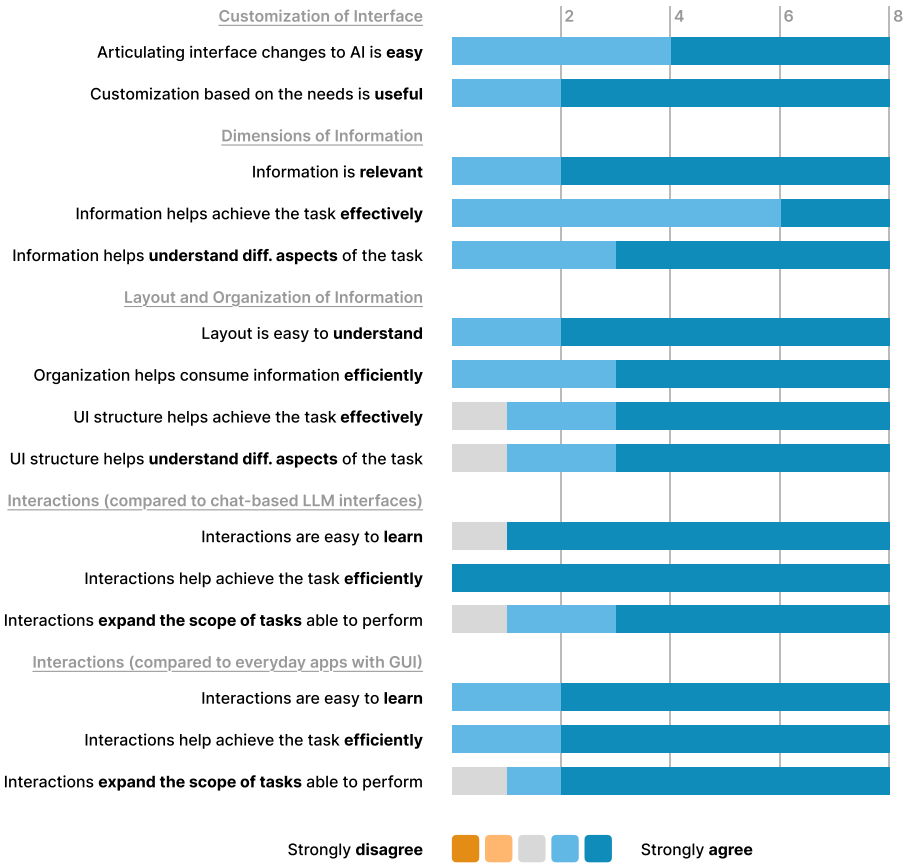


Figure 8: 5-point Likert scale questionnaire questions and user responses.